
Fundamentals Of Spatial Filtering

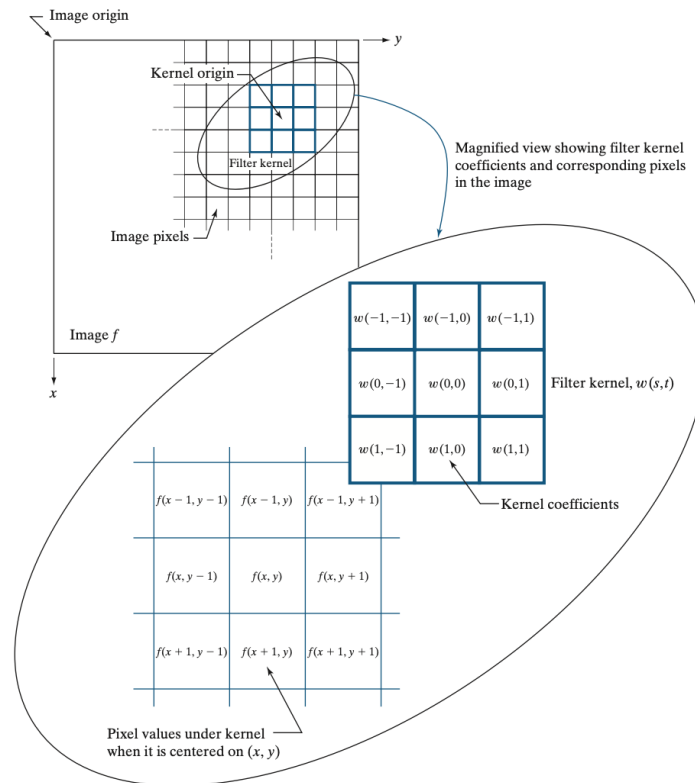
The Mechanics Of Linear Spatial Filtering

- A linear spatial filter performs a sum-of-products operation between an image f and a *filter kernel*, w . The kernel is an array whose size defines the neighborhood of operation, and whose coefficients determine the nature of the filter. Other terms used to refer to a spatial filter kernel are *mask*, *template*, and *window*. We use the term *filter kernel* or simply *kernel*.
- With the pixel at location (x, y) . For a kernel of size $m \times n$, we assume that $m = 2a + 1$ and $n = 2b + 1$, where a and b are nonnegative integers. An image of size $M \times N$ with a kernel of size $m \times n$ is given by the expression

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

The Mechanics Of Linear Spatial Filtering

$$g(x, y) = w(-1, -1)f(x - 1, y - 1) + w(-1, 0)f(x - 1, y) + \dots \\ + w(0, 0)f(x, y) + \dots + w(1, 1)f(x + 1, y + 1)$$



Spatial Correlation And Convolution

- Correlation consists of moving the center of a kernel over an image, and computing the sum of products at each location

$$g(x) = \sum_{s=-a}^a w(s) * f(x + s)$$

- The mechanics of spatial convolution are the same, except that the correlation kernel is rotated by 180°

$$g(x) = \sum_{s=-a}^a w(s) * f(x - s)$$

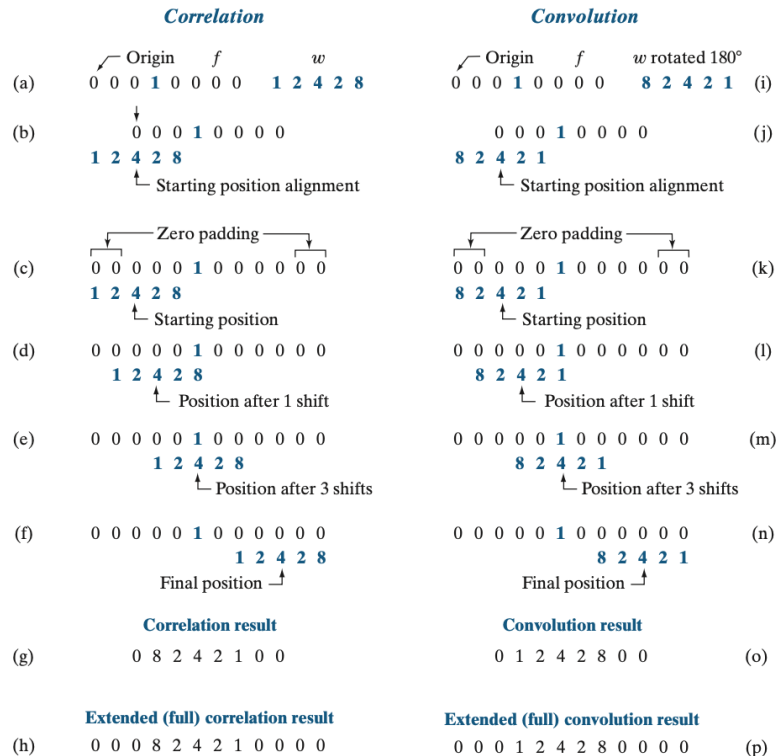
Spatial Correlation And Convolution

- Correlation

$$g(x) = \sum_{s=-a}^a w(s) * f(x + s)$$

- Convolution

$$g(x) = \sum_{s=-a}^a w(s) * f(x - s)$$



Spatial Correlation And Convolution

- Correlation

$$(f \star g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t + \tau)d\tau$$

$$(w \star f)(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t)f(x + s, y + t)$$

- Convolution

$$(f \star g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

$$(w \star f)(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t)f(x - s, y - t)$$

$a = (m - 1) / 2$, $b = (n - 1) / 2$, and we assume that m and n are odd integers.

Origin	f	Padded f
0 0 0 0 0		0 0 0 0 0 0 0
0 0 0 0 0		0 0 0 0 0 0 0
0 0 0 0 0		0 0 0 0 0 0 0
0 0 1 0 0	w	0 0 0 1 0 0 0
0 0 0 0 0	1 2 3	0 0 0 0 0 0 0
0 0 0 0 0	4 5 6	0 0 0 0 0 0 0
0 0 0 0 0	7 8 9	0 0 0 0 0 0 0

(a)

(b)

Initial position for w
1 2 3
4 5 6
7 8 9
0 0 0 1 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

(c)

Correlation result
0 0 0 0 0
0 9 8 7 0
0 6 5 4 0
0 3 2 1 0
0 0 0 0 0

(d)

Full correlation result
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 9 8 7 0 0
0 0 6 5 4 0 0
0 0 3 2 1 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

(e)

Rotated w
9 8 7
6 5 4
3 2 1
0 0 0 1 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

(f)

Convolution result
0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0

(g)

Full convolution result
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 1 2 3 0 0
0 0 4 5 6 0 0
0 0 7 8 9 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

(h)

Spatial Correlation And Convolution

If the kernel and image are of sizes $m \times n$ and $M \times N$, respectively, the padding would have to increase to $(m - 1)$ padding elements above and below the image, and $(n - 1)$ elements to the left and right. Under these conditions, the size of the resulting full correlation or convolution array will be of size $S_v \times S_H$

Spatial Correlation And Convolution

Property	Convolution	Correlation
Commutative	$f \star g = g \star f$	—
Associative	$f \star (g \star h) = (f \star g) \star h$	—
Distributive	$f \star (g + h) = (f \star g) + (f \star h)$	$f \star (g + h) = (f \star g) + (f \star h)$

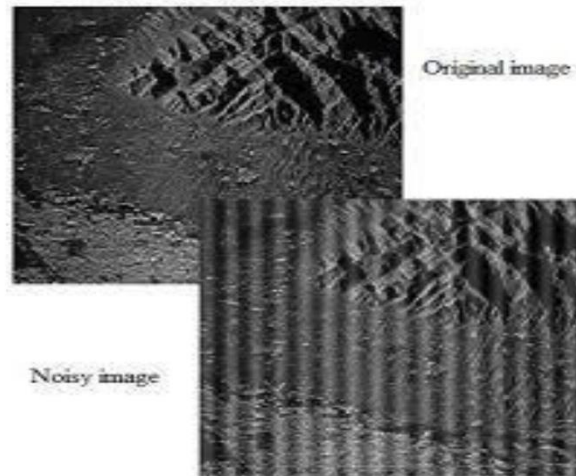
$$F = [1 \ 0 \ -1]$$

$$H = [3 \ 1 \ 4 \ 1 \ 5 \ 9]$$

Calculate F convolution H, H convolution F, F corr H, H corr F with padding.

Noise In Images

- Image noise is random variation of brightness or color information in the captured images.
- It is degradation in image signal caused by external sources.
- Images containing multiplicative noise have the characteristic that brighter the area the noisier it. But mostly it is additive.

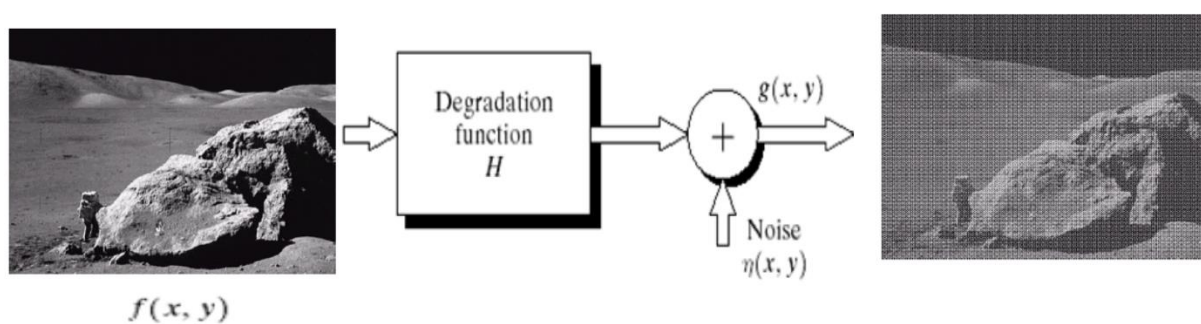


Noise In Images

Model a noisy image as:

$$g(x, y) = f(x, y) + \eta(x, y)$$

Where $g(x, y)$ is function of noisy image, $\eta(x, y)$ is function of image noise, $f(x, y)$ is function of original image.



Sources of Image Noise

- While image being sent electronically from one place to another.
- **Sensor heat** while clicking an image.
- With varying **ISO Factor** which varies with the capacity of camera to absorb light.

Impulse Noise

- Salt Noise: Salt noise is added to an image by addition of random bright (with 255 pixel value) all over the image.
- Pepper Noise: Salt noise is added to an image by addition of random dark (with 0 pixel value) all over the image.
- Salt and Pepper Noise: Salt and Pepper noise is added to an image by addition of both random bright (with 255 pixel value) and random dark (with 0 pixel value) all over the image.



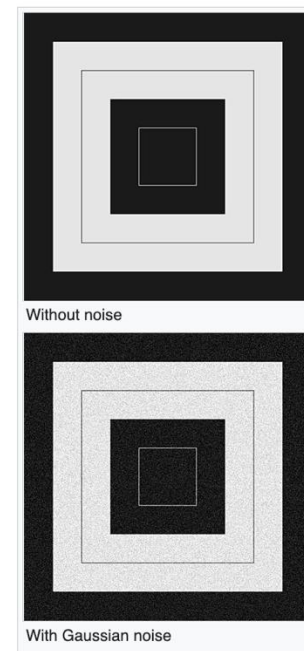
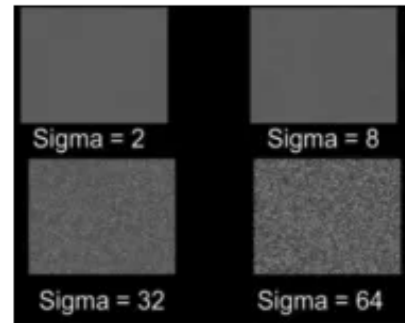
Smoothing (Lowpass) Spatial Filters

Gaussian Noise

- Variations (fluctuation) in intensity drawn from a Gaussian normal distribution
- This noise contains pdf of the normal distribution

$$p(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(z-\mu)^2/2\sigma^2}$$

- Sources of Gaussian Noise:
 - It occurs during acquisition. E.g. Sensor noise caused by poor illumination and/or high temperature
 - Transmission. E.g. Electronic circuit noise



Speckle Noise

- Speckle noise is common in images acquired through ultrasound imaging or synthetic aperture radar (SAR). It appears as granular noise that distorts the underlying structures of the image. It increases the mean grey level of a local area.
- It increases the mean grey level of a local area.



Uniform Noise

- The uniform noise caused by quantizing the pixels of an image to a number of distinct levels is known as quantization noise
- Uniform noise can be analytically described by the gray level values of the noise that are evenly (uniformly) distributed across a specific range
- Quantization noise has an approximately uniform distribution

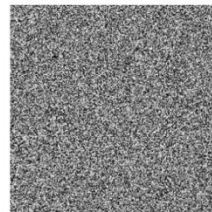
$$p(z) = \begin{cases} \frac{1}{b-a} & \text{if } a \leq z \leq b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}$$

$$\sigma^2 = \frac{(b-a)^2}{12}$$



**Original
Image**



Noisy Image

Noise Removal

Average

- Spatial filter removes impulse noise (salt and pepper noise).
- The averaging filter of size 3x3 pixels is:

$$\hat{f}(x, y) = \frac{1}{mn} \sum_{(s, t) \in S_{xy}} g(s, t)$$

- m, n are no. of rows and no. of columns.
- g(s, t) is observed noisy image.
- Apply noise reduction and enhance the large details.

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Average

The general implementation for filtering an $M \times N$ image with a weighted averaging filter of size $m \times n$ is given

$$g(x, y) = \frac{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)}{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t)}$$

where $m = 2a + 1$, $n = 2b + 1$.

Average

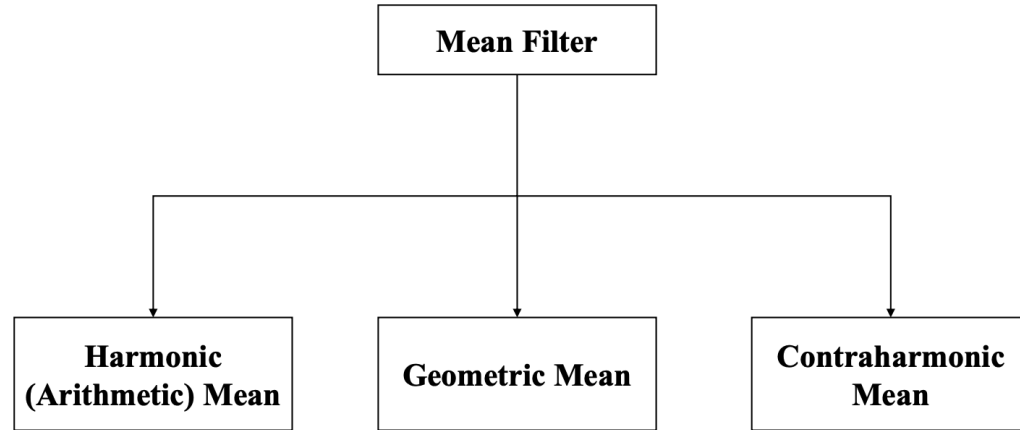
$$\frac{1}{9} \times$$

1	1	1
1	1	1
1	1	1

$$\frac{1}{16} \times$$

1	2	1
2	4	2
1	2	1

Mean filter types



- Arithmetic and geometric mean filters are suitable for Gaussian or uniform noise
- Contraharmonic filters are suitable for impulse noise

Harmonic Mean Filter

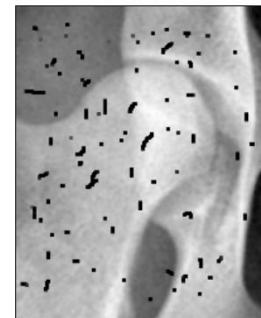
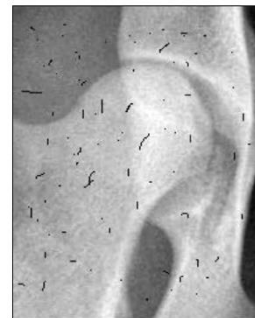
$$\hat{f}(x, y) = \frac{mn}{\sum_{(s, t) \in S_{xy}} \frac{1}{g(s, t)}}$$

- Harmonic mean technique reduces the salt noise.
- But it can't reduce pepper noise well.
- This technique also can reduce damaged image due to Gaussian noise
- The harmonic mean filter is very good at removing positive outliers.

Before:



After:



Harmonic Mean Filter

Consider a 3x3 window of pixel values where there is one positive outlier (255) among other values:

$$\begin{bmatrix} 5 & 10 & 15 \\ 20 & 25 & 255 \\ 30 & 35 & 40 \end{bmatrix}$$

Now, let's calculate the **harmonic mean** for this window.

Harmonic Mean Filter

1. First, we calculate the reciprocal (1 divided by each value):

$$\frac{1}{5}, \frac{1}{10}, \frac{1}{15}, \frac{1}{20}, \frac{1}{25}, \frac{1}{255}, \frac{1}{30}, \frac{1}{35}, \frac{1}{40}$$

These values are approximately:

$$0.2, 0.1, 0.0667, 0.05, 0.04, 0.00392, 0.0333, 0.02857, 0.025$$

2. Next, sum the reciprocals:

$$0.2 + 0.1 + 0.0667 + 0.05 + 0.04 + 0.00392 + 0.0333 + 0.02857 + 0.025 \approx 0.5475$$

3. Now, calculate the harmonic mean:

$$\text{Harmonic Mean} = \frac{9}{0.5475} \approx 16.44$$

So, the harmonic mean for this window is approximately **16.44**, and this value would replace the central pixel (255) in the filtered image.

Harmonic Mean Filter

- Notice that even though there was a large outlier (255), it only had a very small impact on the final harmonic mean (because $\frac{1}{255} \approx 0.00392$, which is much smaller compared to the other values).
- The **smaller values dominate the harmonic mean**, making it effective at reducing the influence of large positive outliers, like the value 255 in this case.

Geometric Mean Filter

$$\hat{f}(x, y) = \left[\prod_{(s, t) \in S_{xy}} g(s, t) \right]^{\frac{1}{mn}}$$

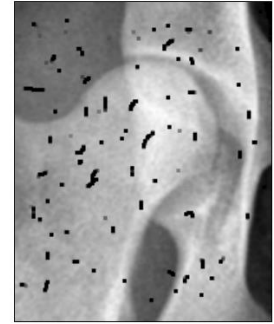
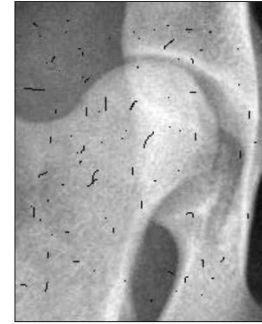
The geometric mean filter is a type of filter that reduces noise while image. It operates by taking the geometric mean of the pixel values in window or kernel) around each pixel.

- The result obtained from geometric mean produces blur image.
- The information of image will lost
- The geometric mean filter is better at removing Gaussian type noise

Before:



After:



Geometric Mean Filter

Let's consider a 3×3 matrix of pixel values, where some of the pixels are corrupted by noise (speckle noise), commonly found in medical or radar images:

$$\begin{bmatrix} 1 & 2 & 100 \\ 3 & 50 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

In this matrix, the value **100** is a **positive outlier** due to noise, which could distort the overall image if we apply filters like the arithmetic mean filter. Now, let's apply the **geometric mean filter**.

Geometric Mean Filter

For a 3×3 window, $n = 9$, so we calculate:

$$\text{Geometric Mean} = (1 \times 2 \times 100 \times 3 \times 50 \times 6 \times 7 \times 8 \times 9)^{\frac{1}{9}}$$

First, we find the product of all pixel values:

$$1 \times 2 \times 100 \times 3 \times 50 \times 6 \times 7 \times 8 \times 9 = 907200000$$

Next, take the 9th root of that product:

$$\text{Geometric Mean} = (907200000)^{\frac{1}{9}} \approx 12.27$$

Thus, the central pixel value will be replaced by approximately **12.27**.

If we use the **arithmetic mean** filter instead, the central pixel will be replaced by the arithmetic average of all the pixel values:

$$\text{Arithmetic Mean} = \frac{1 + 2 + 100 + 3 + 50 + 6 + 7 + 8 + 9}{9} = \frac{186}{9} \approx 20.67$$

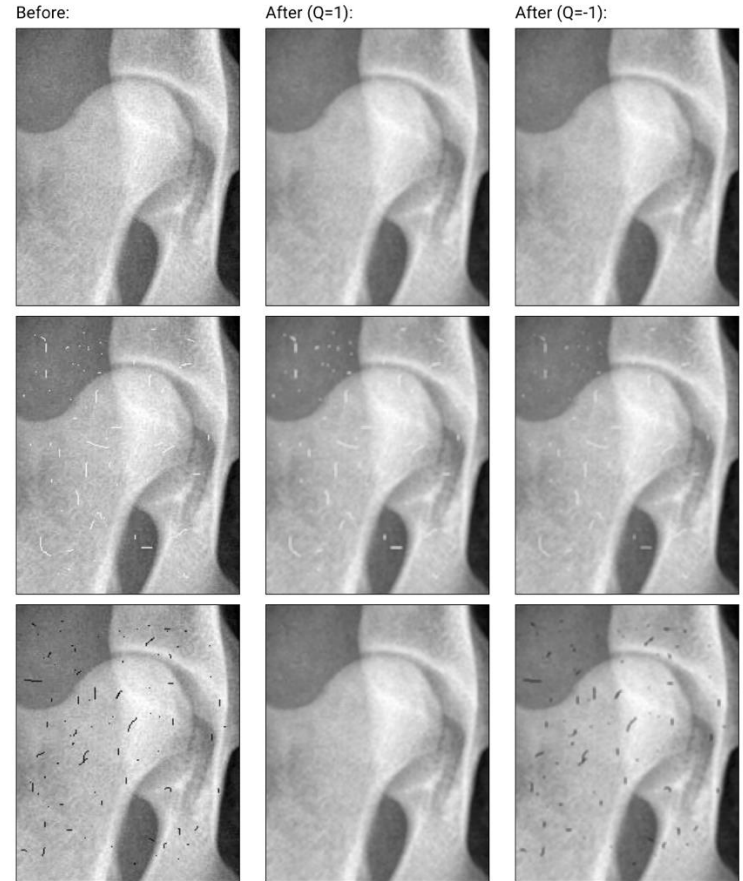
Geometric Mean Filter

- The **geometric mean filter** reduced the impact of the large outlier (100) and replaced the central pixel with **12.27**.
- The **arithmetic mean filter** replaced the central pixel with **20.67**, which was more influenced by the large outlier (100).
- The **Geometric Mean Filter** is very effective at handling **multiplicative noise**, such as speckle noise, especially when there are large positive outliers.
- It reduces noise without distorting the image as much as other filters.
- The **key advantage** is that it is less sensitive to large abnormal values (outliers), preserving important details in the image better.

Contraharmonic Mean Filter

$$\hat{f}(x, y) = \frac{\sum_{(s, t) \in S_{xy}} g(s, t)^{Q+1}}{\sum_{(s, t) \in S_{xy}} g(s, t)^Q}$$

- Q is the order of the filter and adjusting its value changes the filter's behaviour.
- if the Q value is negative value, it can reduce salt noise.
- if the Q value is positive value, it can reduce pepper noise.
- This technique can't eliminate both salt and pepper noise concurrently



Contraharmonic Mean Filter

Suppose you have a 3x3 image matrix (a small portion of an image) affected by **salt noise** (some pixels have been corrupted to the maximum value of 255). The matrix looks like this:

$$\begin{bmatrix} 100 & 150 & 255 \\ 120 & 130 & 125 \\ 255 & 140 & 135 \end{bmatrix}$$

In this case, we have two salt noise values of 255 at positions (1,3) and (3,1).

Contraharmonic Mean Filter

Now let's calculate the contraharmonic mean for the 3x3 submatrix centered at (2,2), with $Q = 1$:

- **Numerator ($Q+1 = 2$):**

$$100^2 + 150^2 + 255^2 + 120^2 + 130^2 + 125^2 + 255^2 + 140^2 + 135^2 = \overset{247300}{10000} + 22500 + 6500$$

- **Denominator ($Q = 1$):**

$$100 + 150 + 255 + 120 + 130 + 125 + 255 + 140 + 135 = 1410$$

Thus:

$$f(2, 2) = \frac{247300}{1410} \approx 175.37$$

Contraharmonic Mean Filter

After applying the filter, the pixel value at the central position (2,2) is updated to approximately **175**. This value helps mitigate the influence of salt noise (the 255 values), making the image smoother and less affected by noise.

If a **mean filter** were applied, the average would be skewed by the extreme noise values (255), whereas the **Contraharmonic Mean Filter** is more effective in reducing noise by adjusting the influence of noisy pixels based on the parameter Q . In this case, it reduces the effect of salt noise, producing a more accurate, clean result.

Order Statistics Filters

Median Filter

- The Median Filter is widely used for removing salt-and-pepper noise (which includes both salt noise, where pixels take on a maximum value like 255, and pepper noise, where pixels take on a minimum value like 0). Unlike the mean filter which calculates the average of the pixel values, the median filter selects the middle value of a sorted list of neighboring pixel values. This makes it effective at preserving edges while removing extreme noise values.

$$\hat{f}(x, y) = \underset{(s, t) \in S_{xy}}{\text{median}} \{g(s, t)\}$$

- Median filter technique reduces impulse noise such as Salt and Pepper well.
- Median filter is defined as:
- Center (median) value in the original image 3x3 pixels is replaced by the median value.

Median Filter

- The 2 D array shows the grayscale image with 3x3 filter.

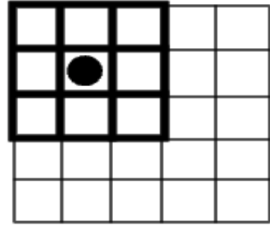
164	156	145	96	168	188
146	135	90	185	200	198
137	83	189	199	214	199
94	191	215	211	201	198
179	221	200	218	222	201
185	210	221	220	198	214

164	156	145	96	168	188
146	135	90	185	200	198
137	83	191	199	214	199
94	189	215	211	201	198
179	221	200	218	222	201
185	210	221	220	198	214

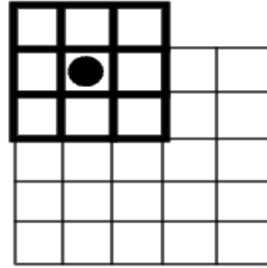
ascending order : 83, 94, 137, 179, **189**, 191, 200, 215, 221

- Pixel intensity value 191 replaced by 3 x 3 filter median intensity value 189

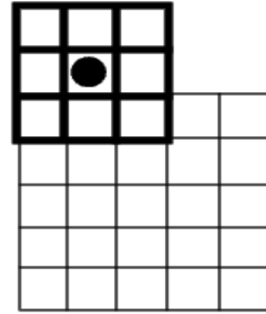
Median Filter



The computational block overlap only pixels that are in the original image.



The computational block overlap pixels outside the original image, but the center pixel overlaps a pixel in the image.



The computational block overlap pixels at the edges only. The center pixel is outside the image.

Median Filter

Consider the same 3x3 image matrix affected by **salt noise** as we used before:

$$\begin{bmatrix} 100 & 150 & 255 \\ 120 & 130 & 125 \\ 255 & 140 & 135 \end{bmatrix}$$

Here, the values 255 at positions (1,3) and (3,1) represent **salt noise**.

Median Filter

To apply the median filter, we consider a 3x3 neighborhood centered at each pixel. Let's focus on the center pixel at position (2,2) (which currently has the value 130). We create a list of the neighboring values:

[100, 150, 255, 120, 130, 125, 255, 140, 135]

Now, we sort this list:

[100, 120, 125, 130, 135, 140, 150, 255, 255]

The **median** of this sorted list is the middle value, which is **135**. Therefore, after applying the median filter, the pixel at position (2,2) will be replaced by 135.

Result:

The updated matrix after applying the median filter to the central pixel becomes:

$$\begin{bmatrix} 100 & 150 & 255 \\ 120 & 135 & 125 \\ 255 & 140 & 135 \end{bmatrix}$$

Median Filter

1. **Resistant to Extreme Values:** The median filter is effective in removing salt-and-pepper noise because it selects the middle value from the surrounding pixels. Extreme values like 255 (salt noise) or 0 (pepper noise) have little influence on the result, as they are pushed to the edges of the sorted list.
2. **Edge Preservation:** Since the median filter replaces the central pixel with a value that is actually present in the neighborhood, it helps preserve edges and important details in the image better than a mean filter, which tends to blur the edges by averaging the pixel values.
3. **Effective Noise Removal:** In cases of salt noise (like 255 in the example), the median filter effectively ignores those extreme values by choosing the central value of the sorted list, thus reducing their impact on the resulting image.

In contrast, filters like the **mean filter** or **Gaussian filter** would average the pixel values, causing the extreme noise values (like 255 or 0) to influence the result more, which may lead to a blurred image. The **median filter** directly addresses this by focusing on the middle value, making it especially suitable for images with salt-and-pepper noise.

Median Filter

Drawbacks of median filter:

- Effective only when the noise is impulse noise (salt and pepper).
- Its output quality deteriorates when the noise is more than 20%.
- It does not work efficiently when the spatial density of noise is high.
- For large kernel size, there is no proper smoothening of the image instead valuable information from the image gets blur.

Adaptive median Filters

- Adaptive median filters can perform better than median filter on impulse noise such as pepper and salt noise.
- The results obtained from adaptive median filter produce slightly smooth image.
- The behaviour of adaptive filters changes depending on the characteristics of the image inside the filter region.
- The median filter performs relatively well on impulse noise as long as the spatial density of the impulse noise is not large.
- The adaptive median filter can handle much more spatially dense impulse noise, and also performs some smoothing for non-impulse noise.
- The key insight in the adaptive median filter is that the filter size changes depending on the characteristics of the image.
- Filtering looks at each original pixel image and generates a new filtered pixel

Adaptive Median Filters

- z_{min} = minimum grey level in S_{xy}
- z_{max} = maximum grey level in S_{xy}
- z_{med} = median of grey levels in S_{xy}
- z_{xy} = grey level at coordinates (x, y)
- S_{max} = maximum allowed size of S_{xy}

Level A: $A1 = z_{med} - z_{min}$
 $A2 = z_{med} - z_{max}$
 if $A1 > 0$ and $A2 < 0$, **Go to** level B
 else increase the window size
 if window size $\leq S_{max}$ level A
 else output z_{med}

Level B: $B1 = z_{xy} - z_{min}$
 $B2 = z_{xy} - z_{max}$
 if $B1 > 0$ and $B2 < 0$, output z_{xy}
 else output z_{med}

Adaptive Median Filters

Goal: The first step is to determine if the **median** value within a window (a region around a pixel) is a good representation of the data in that window.

What happens in Level A?

- You have a small window (let's say 3x3) around the pixel you want to clean.
- Within this window, the filter calculates:
 - $z_{\min}$: The minimum pixel value.
 - $z_{\max}$: The maximum pixel value.
 - z_{med} : The median pixel value.

Adaptive Median Filters

Equations in Level A:

- $A1 = z_{\text{med}} - z_{\text{min}}$: This checks whether the median is greater than the minimum pixel value.
- $A2 = z_{\text{med}} - z_{\text{max}}$: This checks whether the median is smaller than the maximum pixel value.

Condition in Level A:

- If $A1 > 0$ and $A2 < 0$:
 - It means the median value is **between the minimum and maximum values**, which suggests that the median is **not noise**.
 - The algorithm then proceeds to **Level B** to check the central pixel.
- If **either condition fails**:
 - This suggests that the median is **not reliable** (could be noise).
 - The window size is **increased** (e.g., from 3x3 to 5x5), and the process repeats for this larger window.
 - This ensures that the filter considers a larger neighborhood before making a decision. The window can keep increasing until it reaches a pre-defined **maximum size** S_{max} .
 - If the maximum window size is reached and a reliable median is still not found, the filter simply **outputs the median value** of the last window.

Adaptive Median Filters

Level B: Checking if the Central Pixel is Noisy

Goal: After a good median value is found, the next step is to check whether the **central pixel** of the window (the pixel you are trying to clean) is noise.

Equations in Level B:

- $B1 = z_{xy} - z_{\min}$: This checks whether the central pixel is greater than the minimum pixel value.
- $B2 = z_{xy} - z_{\max}$: This checks whether the central pixel is smaller than the maximum pixel value.

Condition in Level B:

- If $B1 > 0$ and $B2 < 0$:
 - It means the central pixel z_{xy} is **not noise**, and the algorithm keeps this original value.
- If **either condition fails**:
 - It means the central pixel is likely noise (salt noise if it's 255 or pepper noise if it's 0).
 - The noisy pixel is replaced by the **median** z_{med} calculated earlier.

Adaptive Median Filters

- **Basic Median Filter:** Uses a fixed window size. If noise is dense, the filter may not clean it well without blurring the image.
- **Adaptive Median Filter:** Adjusts the window size dynamically. If noise is light, the window stays small, preserving details. If noise is heavy, the window grows to clean the noise without affecting the surrounding image quality.

Max and Min Filter

Max Filter:

$$\hat{f}(x, y) = \max_{(s, t) \in S_{xy}} \{g(s, t)\}$$

Min Filter:

$$\hat{f}(x, y) = \min_{(s, t) \in S_{xy}} \{g(s, t)\}$$

Max filter is good for pepper noise and min is good for salt noise

Midpoint Filter

Good for random Gaussian and uniform noise

$$\hat{f}(x, y) = \frac{1}{2} \left[\max_{(s,t) \in S_{xy}} \{g(s,t)\} + \min_{(s,t) \in S_{xy}} \{g(s,t)\} \right]$$

Max filter is good for pepper noise and min is good for salt noise

Midpoint Filter

Advantages of the Midpoint Filter:

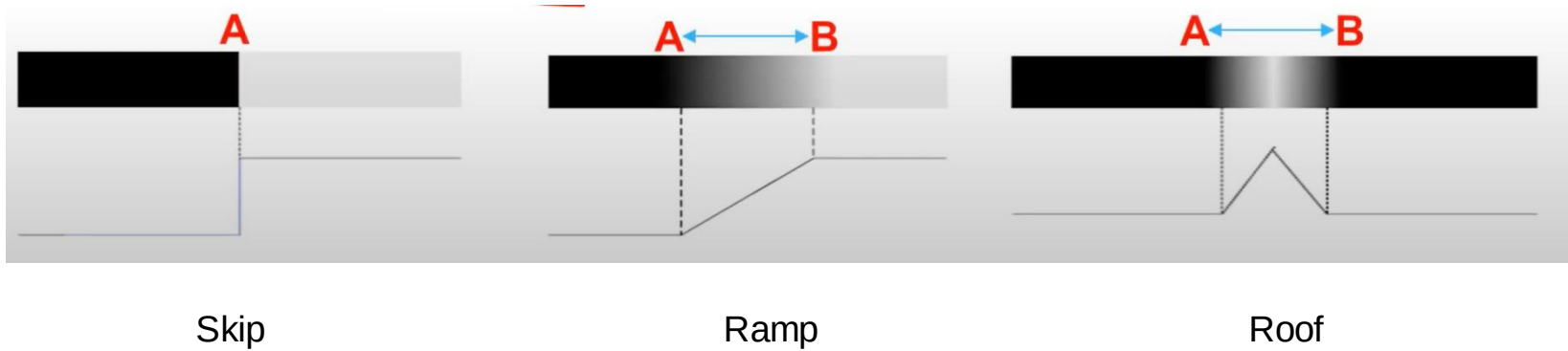
1. **Simple and Fast:** The midpoint filter is computationally efficient, making it suitable for real-time applications.
2. **Effective for Uniform Noise:** It is particularly effective for reducing **random, uniform noise** and small variations in pixel intensity.

Limitations:

- **Not Suitable for Salt-and-Pepper Noise:** Since it only averages the extreme values in a window, it cannot handle **impulsive noise** (salt-and-pepper noise) effectively. The minimum and maximum could be noisy values themselves.
- **Blurring:** Like many smoothing filters, the midpoint filter can cause **blurring** of edges and fine details because it takes an average of extreme values rather than focusing on middle values like a median filter.

Sharpening (Highpass) Spatial Filters

Edge: It is a set of pixels where the gray level (intensity) changes rapidly or suddenly in an image



Sharpening (Highpass) Spatial Filters

A basic definition of the first-order derivative of a one-dimensional function $f(x)$ is the difference

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

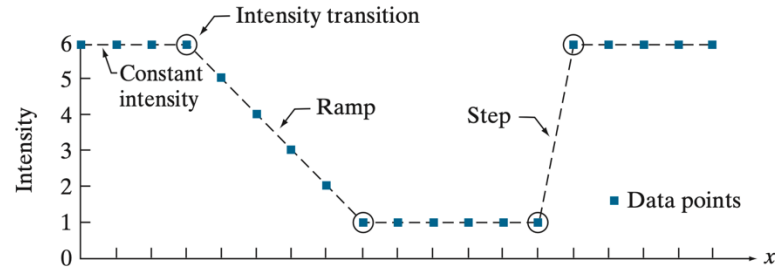
⇒ The difference between adjacent values (the current value compared to the previous value)

We define the second-order derivative of $f(x)$ as the difference

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

⇒ Examining the previous, current, and next values in a sequence

Sharpening (Highpass) Spatial Filters



Values of scan line

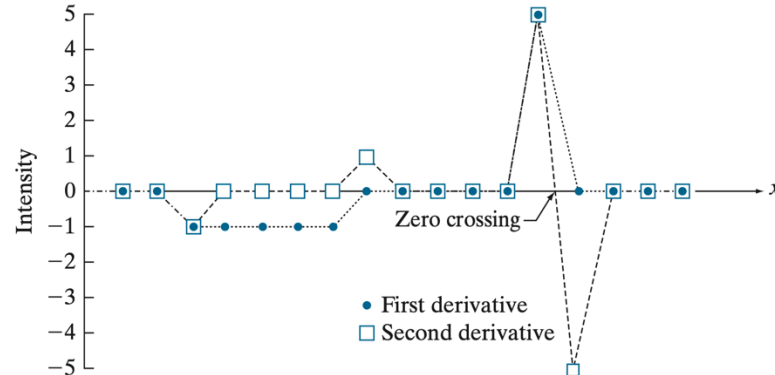
6	6	6	6	5	4	3	2	1	1	1	1	1	1	6	6	6	6	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1st derivative

0	0	-1	-1	-1	-1	-1	0	0	0	0	0	0	5	0	0	0	0
---	---	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---	---

2nd derivative

0	0	-1	0	0	0	0	1	0	0	0	0	0	5	-5	0	0	0
---	---	----	---	---	---	---	---	---	---	---	---	---	---	----	---	---	---



Sharpening (Highpass) Spatial Filters

1st derivative:

- Equal to 0 in areas with constant gray levels.
- Non-zero at the point where it starts transitioning to the ramp boundary and skip boundary region.
- Non-zero in the ramp boundary region.

2st derivative:

- Equal to 0 in regions with constant gray
- Non-zero at the end point of each region.
- Equal to 0 in the ramp boundary region.

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

The gradient of f at the coordinates (x, y) is as follows.

$$\text{grad}(f) = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f(x, y)}{\partial x} = f(x+1, y) - f(x, y) \\ \frac{\partial f(x, y)}{\partial y} = f(x, y+1) - f(x, y) \end{bmatrix}$$

The magnitude of this vector is the rate of change of the value at the coordinates (x, y) in the direction of the gradient.

$$\nabla f(x, y) = \text{mag}(\nabla f) = \sqrt{G_x^2 + G_y^2} = \sqrt{\left(\frac{\partial f(x, y)}{\partial x}\right)^2 + \left(\frac{\partial f(x, y)}{\partial y}\right)^2}$$

$$\nabla f \approx |G_x| + |G_y|$$

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

From the formula of the gradient of f at the coordinates (x, y)

$$\nabla f \approx |G_x| + |G_y|$$

$$G_x = \frac{\partial f(x, y)}{\partial x} = f(x+1, y) - f(x, y)$$

The change of the subsequent element adjacent to the previous element in the x direction.

$$G_y = \frac{\partial f(x, y)}{\partial y} = f(x, y+1) - f(x, y)$$

The change of the subsequent element adjacent to the previous element in the y direction.

Robert proposed a kernel 2x2 as:

$f(x, y)$	$f(x+1, y)$
$f(x, y+1)$	$f(x+1, y+1)$

$$|G_{cross1}| = |f(x+1, y+1) - f(x, y)|$$

$$|G_{cross2}| = |f(x, y+1) - f(x+1, y)|$$

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

From the formula of the gradient of f at the coordinates (x, y)

$$\nabla f \approx |G_x| + |G_y|$$

$$G_x = \frac{\partial f(x, y)}{\partial x} = f(x+1, y) - f(x, y)$$

The change of the subsequent element adjacent to the previous element in the x direction.

$$G_y = \frac{\partial f(x, y)}{\partial y} = f(x, y+1) - f(x, y)$$

The change of the subsequent element adjacent to the previous element in the y direction.

Robert proposed a kernel 2x2 as:

$f(x, y)$	$f(x+1, y)$
$f(x, y+1)$	$f(x+1, y+1)$

$$|G_{cross1}| = |f(x+1, y+1) - f(x, y)|$$



-1	0
0	1

$$|G_{cross2}| = |f(x, y+1) - f(x+1, y)|$$



USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

From the formula of the gradient of f at the coordinates (x, y)

$$\nabla f \approx |G_x| + |G_y|$$

$$G_x = \frac{\partial f(x, y)}{\partial x} = f(x+1, y) - f(x, y)$$

The change of the subsequent element adjacent to the previous element in the x direction.

$$G_y = \frac{\partial f(x, y)}{\partial y} = f(x, y+1) - f(x, y)$$

The change of the subsequent element adjacent to the previous element in the y direction.

Robert proposed a kernel 2x2 as:

$f(x, y)$	$f(x+1, y)$
$f(x, y+1)$	$f(x+1, y+1)$

$$|G_{cross1}| = |f(x+1, y+1) - f(x, y)|$$



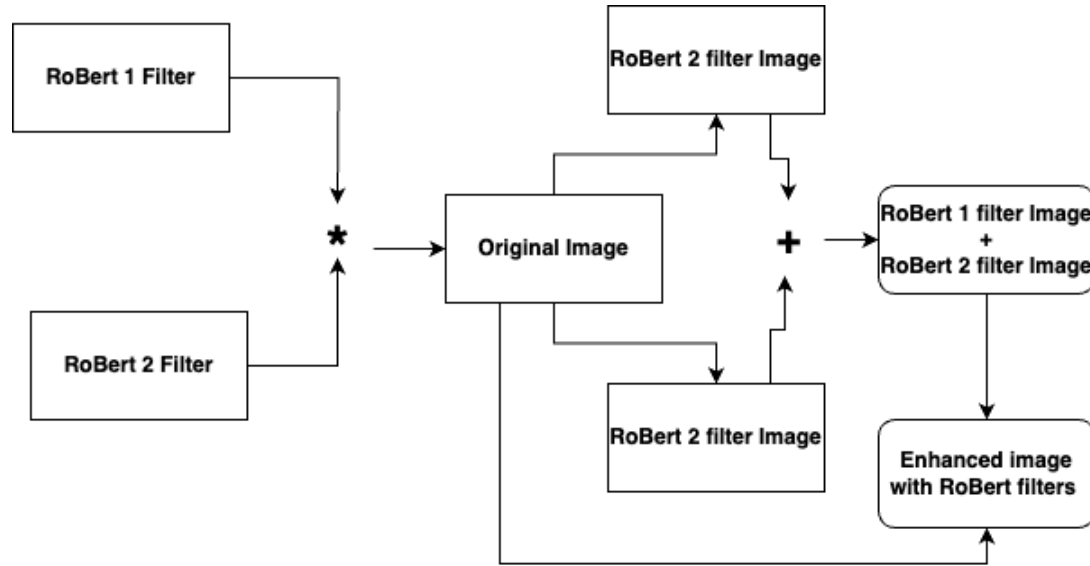
-1	0
0	1

$$|G_{cross2}| = |f(x, y+1) - f(x+1, y)|$$

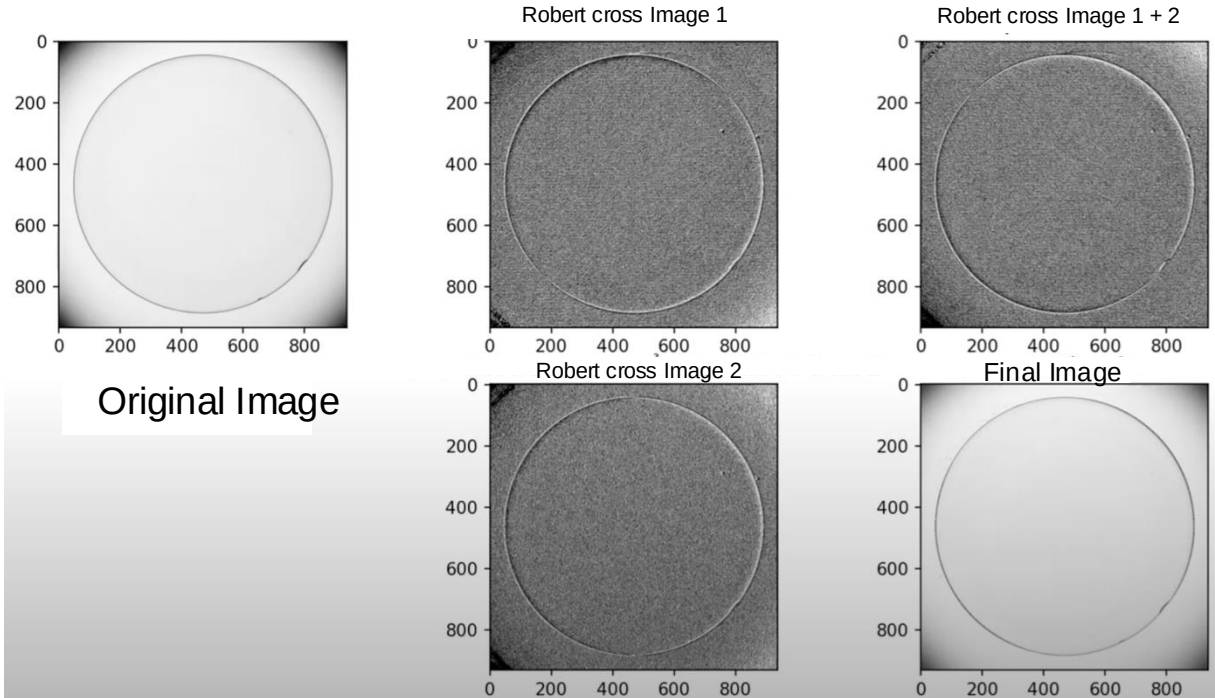


0	-1
1	0

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT



USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT



USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

SoBel Filter

$$|G_X| = |f(x-1, y+1) + 2f(x, y+1) + f(x+1, y+1) - (f(x-1, y-1) + 2f(x, y-1) + f(x+1, y-1))|$$

$f(x-1, y-1)$	$f(x, y-1)$	$f(x+1, y-1)$
$f(x-1, y)$	$f(x, y)$	$f(x+1, y)$
$f(x-1, y+1)$	$f(x, y+1)$	$f(x+1, y+1)$

-1	-2	-1
0	0	0
1	2	1

$$|G_Y| = |f(x+1, y-1) + 2f(x+1, y) + f(x+1, y+1) - (f(x-1, y-1) + 2f(x-1, y) + f(x-1, y+1))|$$

-1	0	1
-2	0	2
-1	0	1

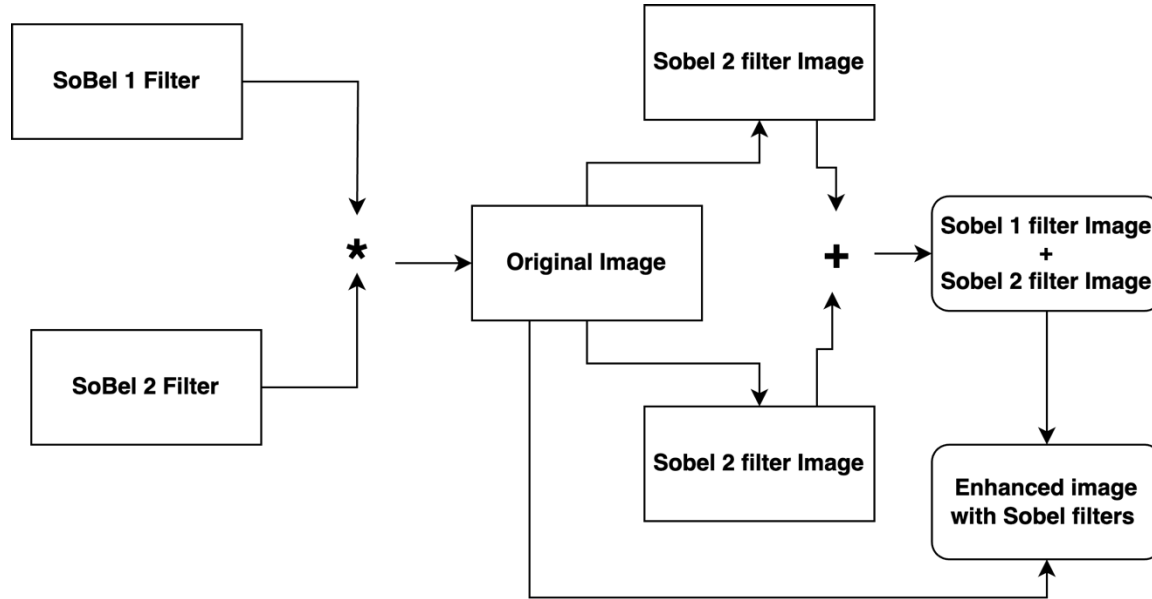
USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

SoBel Filter

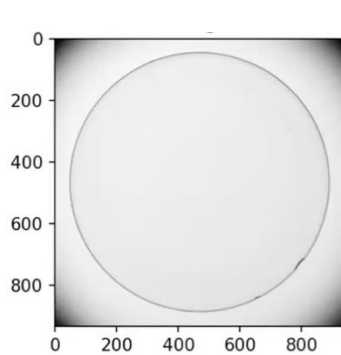
-1	-2	-1	-1	0	1
0	0	0	-2	0	2
1	2	1	-1	0	1

- The sum of the elements surrounding the central point is zero.
- To filter the image, we use both operators and combine (add) the results together.

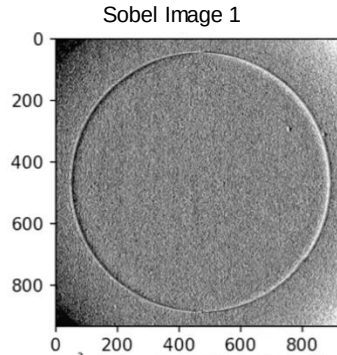
USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT



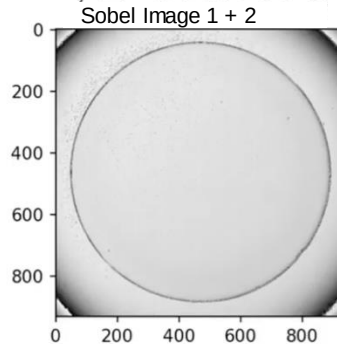
USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT



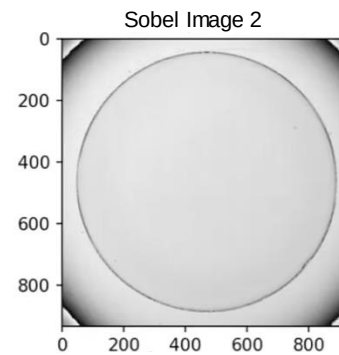
Original Image



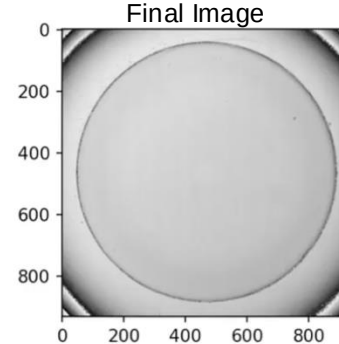
Sobel Image 1



Sobel Image 1 + 2



Sobel Image 2



Final Image

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

It can be shown that the simplest isotropic derivative operator (kernel) is the Laplacian, which, for a function (image) $f(x, y)$ of two variables, is defined as

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

In the x-direction:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

In the y-direction:

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

It can be shown that the simplest isotropic derivative operator (kernel) is the Laplacian, which, for a function (image) $f(x, y)$ of two variables, is defined as

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

In the x-direction:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

In the y-direction:

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

It follows from the preceding three equations that the discrete Laplacian of two variables is:

$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

$f(x-1, y-1)$	$f(x, y-1)$	$f(x+1, y-1)$
$f(x-1, y)$	$f(x, y)$	$f(x+1, y)$
$f(x-1, y+1)$	$f(x, y+1)$	$f(x+1, y+1)$

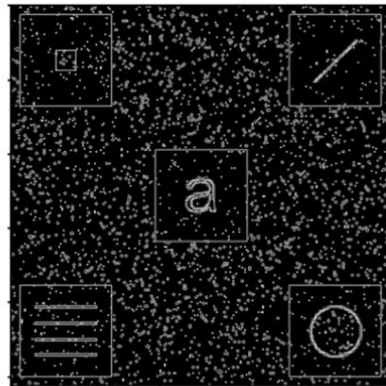
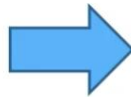
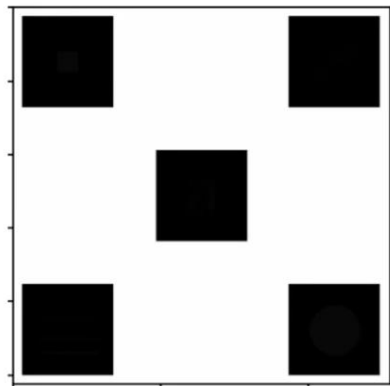
Original Image

0	1	0
1	-4	1
0	1	0

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

It follows from the preceding three equations that the discrete Laplacian of two variables is:

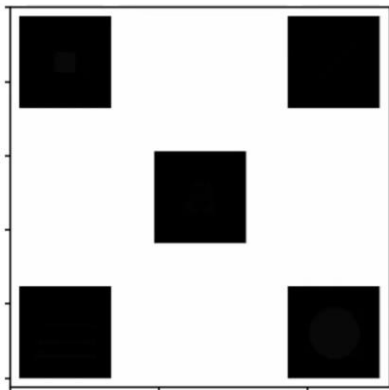
$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$



USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

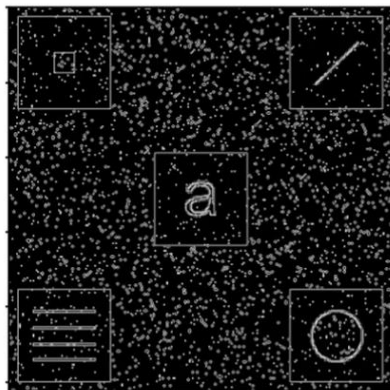
•

$$f(x, y) - \nabla^2 f = g(x, y)$$

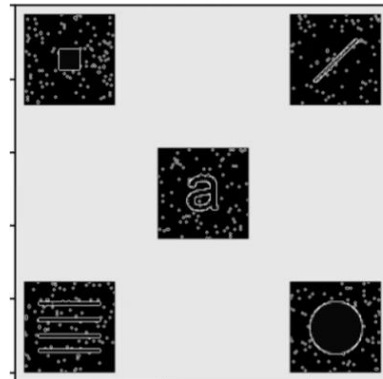


Original Image

-



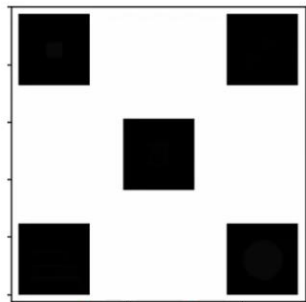
=



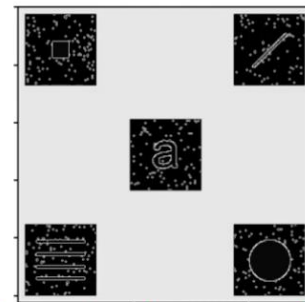
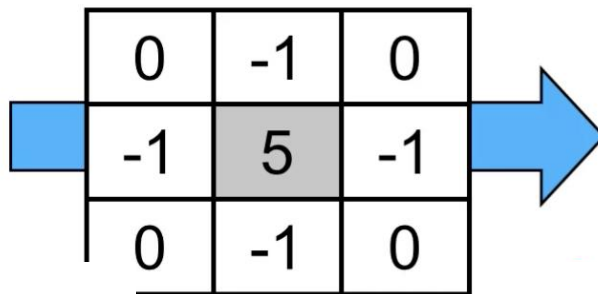
USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

Simple Formula of Laplacian

$$\begin{aligned} \underline{g(x, y)} &= f(x, y) - \nabla^2 f \\ &= f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)] \\ &= 5f(x, y) - f(x+1, y) - f(x-1, y) - f(x, y+1) - f(x, y-1) \end{aligned}$$



Original Image



Final Image

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

First Variant of the Laplacian

$f(x-1,y-1)$	$f(x,y-1)$	$f(x+1,y-1)$
$f(x-1,y)$	$f(x,y)$	$f(x+1,y)$
$f(x-1,y+1)$	$f(x,y+1)$	$f(x+1,y+1)$

Original Image

0	1	0
1	-4	1
0	1	0

+

1	0	1
0	-4	0
1	0	1

=

1	1	1
1	-8	1
1	1	1

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

Second Variant of the Laplacian

$f(x-1,y-1)$	$f(x,y-1)$	$f(x+1,y-1)$
$f(x-1,y)$	$f(x,y)$	$f(x+1,y)$
$f(x-1,y+1)$	$f(x,y+1)$	$f(x+1,y+1)$

Original Image

0	1	0
1	-4	1
0	1	0

====>

0	-1	0
-1	4	-1
0	-1	0

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

Third Variant of the Laplacian

$f(x-1,y-1)$	$f(x,y-1)$	$f(x+1,y-1)$
$f(x-1,y)$	$f(x,y)$	$f(x+1,y)$
$f(x-1,y+1)$	$f(x,y+1)$	$f(x+1,y+1)$

Original Image

1	1	1
1	-8	1
1	1	1

====>

-1	-1	-1
-1	8	-1
-1	-1	-1

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

All Variants of the Laplacian

0	1	0
1	-4	1
0	1	0



1	1	1
1	-8	1
1	1	1

0	-1	0
-1	4	-1
0	-1	0

-1	-1	-1
-1	8	-1
-1	-1	-1

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

All Variants of the Laplacian

0	1	0
1	-4	1
0	1	0



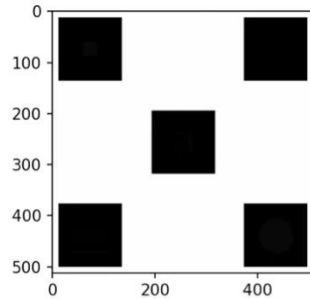
1	1	1
1	-8	1
1	1	1

0	-1	0
-1	4	-1
0	-1	0

-1	-1	-1
-1	8	-1
-1	-1	-1

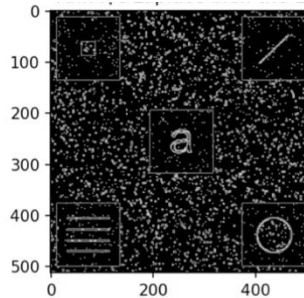
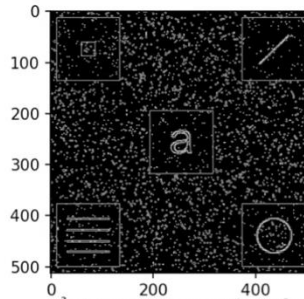
0	-1	0
-1	5	-1
0	-1	0

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT



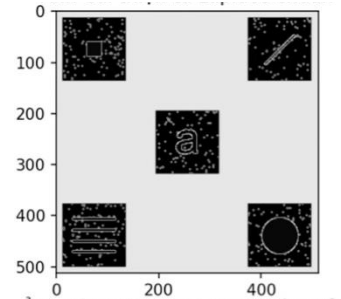
Original Image

Standard Laplacian Filter Image



Laplacian Filter Image - Variant 1

Enhanced Image from Standard Laplacian



Enhanced Image from Laplacian - Variant 1

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

First-Order Derivative

Definition: Measures the rate of change in pixel intensity; calculates the gradient.

Application: Primarily used for edge detection (e.g., Sobel, Prewitt filters).

Information Provided: Identifies edge direction and intensity, but not curvature.

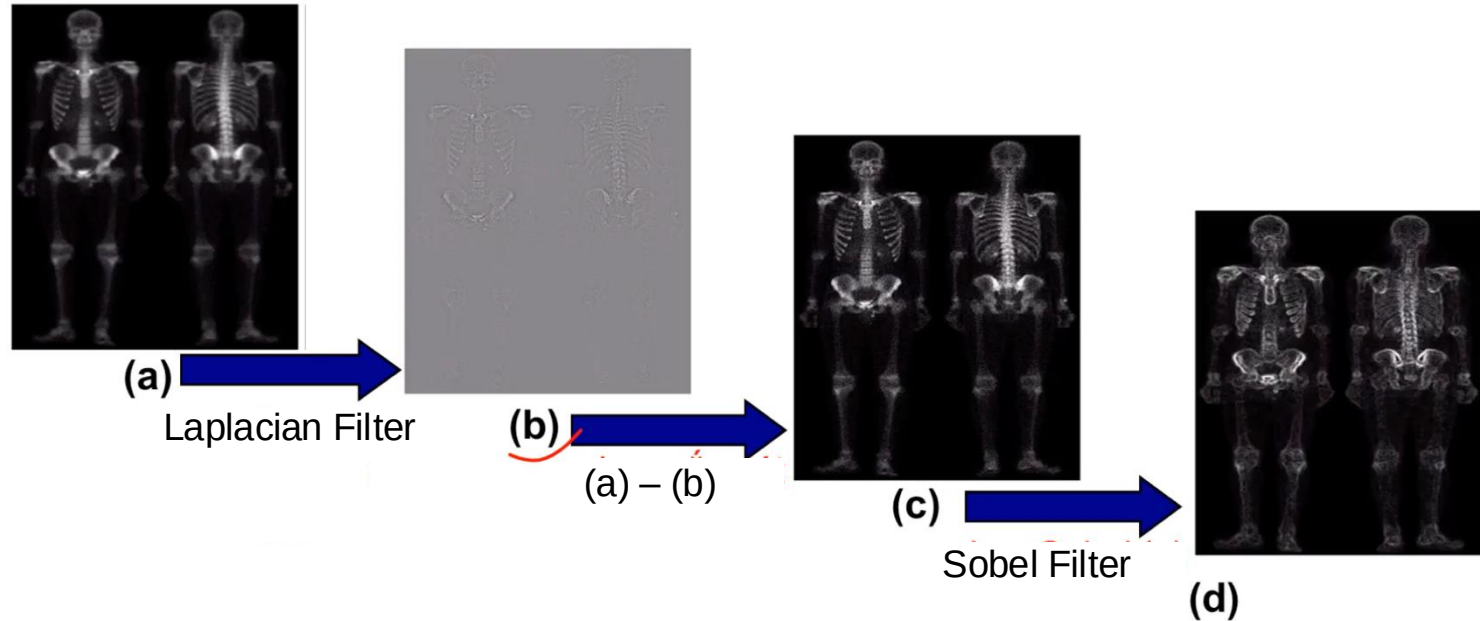
Second-Order Derivative

Definition: Measures the change of the gradient, indicating curvature.

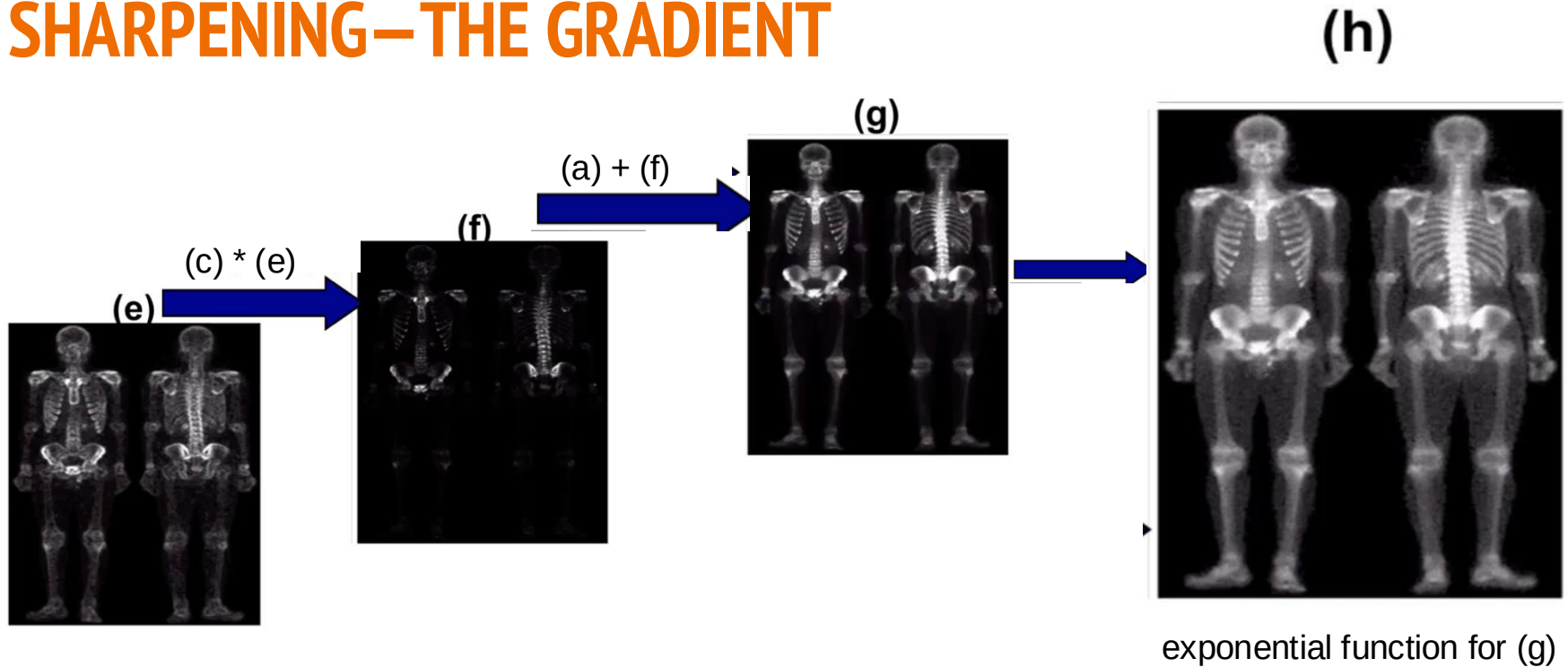
Application: Used for detecting features like corners and edges with curvature (e.g., Laplacian filter).

Information Provided: Provides information on curvature and complex features in images.

USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT



USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT



USING FIRST-ORDER DERIVATIVES FOR IMAGE SHARPENING—THE GRADIENT

